

# How Fast Are You Going Growing?

Yayati Gupta

February 6, 2026 (10:35–11:30)

## Contents

|                                                                   |          |
|-------------------------------------------------------------------|----------|
| <b>1 In Previous Class</b>                                        | <b>1</b> |
| <b>2 Today</b>                                                    | <b>1</b> |
| 2.1 Binary Search with Recursion [A single chain of calls]        | 1        |
| 2.2 What makes you better?                                        | 3        |
| 2.2.1 The street rules of growth                                  | 3        |
| 2.2.2 Common growth functions in decreasing order of growth rate: | 3        |
| 2.3 Analysing Linear Search                                       | 5        |
| <b>3 Coming next</b>                                              | <b>5</b> |
| 3.1 Big-Oh, Theta, Omega notations                                | 5        |

## 1 In Previous Class

1. Comparison of running times for linear and binary search
2. Recursion

## 2 Today

### 2.1 Binary Search with Recursion [A single chain of calls]

Algorithm

```
int BS(int A[], int s, int e, int key)
{
    if (s > e) return -1;
    if A[(s+e)/2] == key return (s+e)/2;
    else if key < A[(s+e)/2]
        return (A, s, (s+e)/2 - 1, key)
    else
        return (A, (s+e)/2 + 1, e, key)
}
```

returns(8)

Dry Run

Say, Searching for 46, key = 46)

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |     |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|-----|
| 3 | 8 | 12 | 17 | 23 | 29 | 34 | 40 | 46 | 53 | 61 | 70 | 82 | 95 | 109 |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|-----|

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

BS(A, 0, 14, 46)

$$m = 7$$

$$A[7] = 40$$

$$46 > 40$$

return BS(A, 8, 14, 46)

$$m = \frac{8+14}{2} = 11$$

$$A[11] = 70$$

$$70 > 46$$

return BS(A, 8, 10, 46)

$$m = 9$$

$$A[9] = 53$$

$$46 < 53$$

return BS(A, 8, 8, 46)

$$m = \frac{8+8}{2} = 8$$

$$A[8] = 46$$

return 8

## 2.2 What makes you better?

### 2.2.1 The street rules of growth

#### What Doesn't Matter

- Adding or subtracting constants
- Multiplying by constants
- Lower-order terms
- Choice of logarithm base  $\log_2 n, \log_{10} n, \ln n$
- Small input behavior — asymptotics care about *large*  $n$

#### What Does Matter:

- The fastest-growing term
- The *shape* of the function as  $n \rightarrow \infty$
- The growth category: constant,  $\log n$ ,  $n$ ,  $n \log n$ ,  $n^2$ ,  $2^n$ , etc.
- Whether growth is polynomial vs exponential
- How the function scales when input size doubles

### 2.2.2 Common growth functions in decreasing order of growth rate:

- |                  |                                 |
|------------------|---------------------------------|
| 1. $n!$          | (Factorial growth)              |
| 2. $c^n (c > 1)$ | (Exponential growth)            |
| 3. $n^k (k > 2)$ | (High-degree polynomial growth) |
| 4. $n^2$         | (Quadratic growth)              |
| 5. $n \log n$    | (Linearithmic growth)           |
| 6. $n$           | (Linear growth)                 |
| 7. $\log n$      | (Logarithmic growth)            |
| 8. 1             | (Constant growth)               |

**Group the following functions by their growth shape (for very large  $x$ )**

Below are several functions. Your task is to mentally compare how they grow and group together the ones that have the **same overall shape of growth**.

$$x \quad 7x + 100 \quad \log x \quad \log(10x)$$

$$x^2 \quad 3x^2 + 5x \quad x^3 \quad 4x^3 - x$$

$$\sqrt{x} \quad x \log x \quad 2^x \quad 5^x$$

$$100 \quad 50 \log x + x \quad x^4 \quad x^2 \log x$$

**Write your groups here (same shape together):**

Group 1:  $x, 7x + 100, 50 \log x + x$

Group 2:  $\log x, \log(10x)$

Group 3:  $x^2, 3x^2 + 5x$

Group 4:  $x^3, 4x^3 - x$

Group 5:  $\sqrt{x}$

Group 6:  $x \log x$

**What mental rule did you use to decide the groups?**

G7:  $2^x$

G8:  $5^x$

G9: 100

G10:  $x^4$

G11:  $x^2 \log x$

## 2.3 Analysing Linear Search

### Linear Search (Using Flag)

```
LinearSearch(A, n, key)
  found ← false → c
  for i ← 0 to n-1 do
    if A[i] = key then ] c
      found ← true
      break
    end if
  end for

  if found = true then c
    return i
  else
    return -1 c
  end if
```

## 3 Coming next

### 3.1 Big-Oh, Theta, Omega notations

Loop repeated n times  
in worst case

$$\Rightarrow c + nc + 2c$$
$$= c(n+3)$$

### Linear Search (Direct Return)

```
LinearSearch(A, n, key)
  for i ← 0 to n-1 do
    if A[i] = key then ] c
      return i
    end if
  end for
```

return -1 → c

worst case

Loop repeated n times

$$nc + c$$

$$c(n+1)$$